Beyond Hallucination: A System-Level Failure Taxonomy for Production LLMs

Priyanka Bajaj
Independent Researcher
https://github.com/priyanka25aug/llm-failure-taxonomy

Abstract

Large language models are increasingly deployed in production systems serving millions of users across regulated, high-stakes domains. Existing failure analysis focuses predominantly on model-level errors — hallucination rates, factuality benchmarks, and safety refusal classification. This framing, while necessary, is insufficient for production engineers who must reason about system reliability across the full deployment stack.

We present the first system-level failure taxonomy for production LLM deployments, comprising six classes: *Model Drift*, *Infrastructure*, *Integration*, *Evaluation*, *Safety & Compliance*, and *Operational*. Unlike model-centric taxonomies, our classification scheme captures failures that originate outside the model weights — in serving infrastructure, RAG retrieval pipelines, evaluation frameworks, and operational processes — and classifies them by their detectability profile and blast radius.

We validate the taxonomy against 50 real-world production incidents drawn from court documents, regulatory filings, academic papers, and public postmortems. Key findings: 50% of incidents were *silent* (no immediate alert; detected via audit or user complaint); Safety & Compliance and Integration failures each account for 26% of incidents; and Infrastructure failures, while less frequent, show the shortest mean time to detection.

We further introduce a *per-use-case failure budget framework* — a formal model for assigning acceptable failure rates by use case risk class (Decision-Critical, Customer-Facing, Internal Productivity, Experimental) rather than by model accuracy alone. This reframing shifts engineering teams from debating "which model is best" to reasoning about acceptable risk envelopes per workflow. All data, code, and the taxonomy schema are released at https://github.com/priyanka25aug/llm-failure-taxonomy.

1 Introduction

1.1 Three Failures That Look Like One

In 2023, a New York attorney submitted a legal brief to a federal court containing citations to six non-existent cases. All six had been generated by ChatGPT. The sanctions opinion in *Mata v. Avianca* [21] documented the first high-profile instance of an LLM hallucination causing demonstrable harm in a professional setting. The press called it an AI failure. Technically, it was: the model fabricated legal authority that did not exist.

That same year, Air Canada operated a customer service chatbot that told a grieving passenger that he could apply for a bereavement fare discount retroactively — a policy that does not and never did exist [5]. The British Columbia Civil Resolution Tribunal found Air Canada liable. The model may have produced a factually coherent sentence. The failure was in the system around it: no guardrail verified chatbot claims against the live policy database; no escalation path existed when the chatbot entered an unknown policy domain. This is not a hallucination. This is an integration failure.

The UK Post Office Horizon system [17], in operation from 1999 onward, produced accounting discrepancies that led to the wrongful prosecution of more than 700 sub-postmasters. The root cause was not a model producing wrong outputs. It was an operational failure: no independent audit mechanism, no escalation when system

outputs contradicted operator experience, and a governance structure that treated system output as ground truth. This pattern — where the operational envelope around an automated system is deficient, not the system itself — maps directly to what we term *Operational Failures* in our taxonomy.

All three incidents are frequently cited as "AI failures." Only the first is a model-layer failure in any precise sense. The second is an integration failure. The third is an operational failure. Treating them as equivalent produces misdiagnosis, and misdiagnosis produces the wrong remediation.

1.2 The Gap in Existing Frameworks

The machine learning systems literature has produced a series of foundational works on production failures. Sculley et al. [19] identified hidden technical debt patterns in ML systems — notably entanglement, feedback loops, and undeclared consumers — but predated the deployment of large language models at consumer scale. Amershi et al. [2] surveyed software engineering challenges in ML at Microsoft, identifying challenges in data collection, model evaluation, and deployment, but their classification scheme does not capture LLM-specific failure modes. Paleyes et al. [15] surveyed ML deployment challenges through 2021, again without addressing the LLM-specific stack.

The LLM-specific literature has progressed rapidly but remains focused on model-layer phenomena. Hallucination surveys [10, 11] provide detailed taxonomies of model output errors. Adversarial input research [9, 16, 22] addresses one sub-class of what we term Integration Failures. Safety alignment work [3] addresses one sub-class of Safety & Compliance failures. Evaluation critiques [4, 13, 18] address one sub-class of Evaluation Failures. No prior work unifies these into a system-level classification with detectability profiles, blast radius characterisation, and a formal framework for acceptable failure rates.

1.3 Contributions

This paper makes the following contributions:

1. A **six-class system-level failure taxonomy** with sub-classes, detectability profiles, and blast radius characterisation — designed to be actionable by production engineering teams, not just model researchers.
2. A **labeled dataset of 50 real-world incidents** drawn from verifiable public sources (court documents, regulatory filings, academic papers, public postmortems), with full provenance and label methodology.
3. A **per-use-case failure budget framework** formalising acceptable failure rates by risk class, and a reference implementation for computing failure budget utilisation in production.
4. A **reference rule-based classifier** for automated incident triage, evaluated against the labeled dataset with per-class precision and recall.

Section 2 reviews related work and identifies the coverage gap. Section 3 introduces the taxonomy design principles and classification dimensions. Section 4 defines each of the six classes. Section 5 presents the failure budget framework. Section 6 describes dataset construction and classifier validation. Sections 7 and 8 discuss implications and open problems.

2 Related Work

2.1 ML System Reliability

The foundational contribution of Sculley et al. [19] introduced the concept of *technical debt* in ML systems, identifying entanglement, hidden feedback loops, undeclared consumers, and unstable data dependencies as recurring failure patterns. These patterns remain relevant for LLM systems, but the taxonomy predates the modern deployment stack entirely: it does not address serving infrastructure for large models, RAG pipelines, prompt injection, or the compliance requirements introduced by the EU AI Act [7] and the Digital Operational Resilience Act [6].

Amershi et al. [2] identified three software engineering challenges specific to ML: data collection and management, model building, and deployment monitoring. Their framework applies to supervised learning systems and does not address the non-determinism, prompt sensitivity, or context-window constraints of autoregressive LLMs.

Paleyes et al. [15] surveyed 35 case studies of ML deployment, identifying data, model, code,

and organisational failure categories. Their review covers pre-LLM deployments and lacks the RAG, tool-call, and safety-compliance subclasses relevant to contemporary production systems.

2.2 LLM Failure Analysis

Hallucination surveys [10, 11] provide the most rigorous model-layer failure taxonomies available, distinguishing intrinsic and extrinsic hallucinations, faithfulness failures, and factuality failures. These are essential for understanding model-layer behaviour but do not address failures that originate in the system around the model.

Adversarial input research [16, 22] demonstrates that LLM safety training fails under distribution shift, and that external inputs can override system prompts. Greshake et al. [9] introduced indirect prompt injection, where adversarial instructions are embedded in retrieved content processed by the model. These contributions map to a single sub-class (3a) of our CLASS_3 Integration taxonomy.

Constitutional AI [3] and related alignment work address the training-time mitigation of one subclass (5c) of our Safety & Compliance failures: policy boundary violations. The broader compliance failure surface — PII leakage, hallucinated legal citations, auditability gaps, copyright reproduction — is not addressed by alignment training alone.

Evaluation critiques [4, 13, 18, 20] collectively document the failure of benchmark proxies to capture real-world model behaviour. HELM [13] explicitly acknowledges the "evaluation gap" between benchmark performance and production reliability. These contributions map to our CLASS_4 Evaluation Failures, which we treat as a system-level failure class in their own right.

2.3 AI Incident Databases

The AI Incident Database [14] provides the most comprehensive public collection of real-world AI failures, with over 600 indexed incidents at the time of writing. The AIAAIC repository [1] provides a complementary collection with stronger regulatory context. Both databases collect *incidents* without providing a structured failure taxonomy with detectability profiles or formal failure budget formalism. This paper provides the structured layer on top of the incident collection tradition.

2.4 Coverage Gap

Table 1 maps prior work to our six failure classes, demonstrating that no single prior work covers all classes. The Coverage row counts how many of the six classes each work addresses fully.

3 Taxonomy Framework

3.1 Design Principles

Four principles guide the taxonomy design.

Layer-specificity.. Each failure class maps to a distinct system layer: model weights and inference (C1), serving infrastructure (C2), application–LLM integration (C3), evaluation and observability (C4), governance and compliance (C5), and operational process (C6). Layer-specific classification enables routing each failure to the appropriate owning team rather than treating all failures as model problems.

Actionability.. Each class implies a distinct remediation path. Infrastructure failures require capacity planning and load-shedding; integration failures require guardrails and output validators; operational failures require runbook creation and monitoring coverage. A taxonomy that cannot drive remediation produces correct labels but no operational value.

Detectability as a first-class attribute.. We classify not only *what* fails but *when and how* it becomes observable. Detectability takes three values: *immediate* (alert fires within minutes), *delayed* (discovered within hours to days), or *silent* (discovered only via audit, user complaint, or manual review, weeks to months later). Our key empirical finding — that 50% of incidents are silent — motivates this as a first-class attribute.

Blast radius.. Scope of impact is a classification attribute, not an afterthought. Blast radius takes six levels: `single_request` → `single_user` → `user_cohort` → `team` → `org_wide` → `public`. The same failure class (e.g. prompt injection) can have very different operational impact depending on whether it affects one request or all requests from a user cohort.

3.2 Classification Dimensions

Table 2 summarises the seven classification dimensions applied to each labeled incident.

Table 1. Coverage of six failure classes in prior work. ✓ = full coverage; ∼ = partial; ✗ = not addressed.

Work	C1 Drift	C2 Infra	C3 Integr.	C4 Eval	C5 Safety	C6 Ops	/6
Sculley et al. 2015	∼	∼	✗	✗	✗	∼	1.5
Amershi et al. 2019	∼	✗	✗	∼	✗	∼	1.5
Paleyes et al. 2022	∼	∼	✗	✗	✗	∼	1.5
Ji et al. 2023	✓	✗	✗	✗	∼	✗	1.5
Wei et al. 2023	✗	✗	∼	✗	∼	✗	1.0
Liang et al. 2022	✗	✗	✗	✓	✗	✗	1.0
AIID 2021	∼	∼	∼	✗	∼	∼	2.0
Weidinger et al. 2021	✗	✗	∼	✗	✓	✗	1.5
This work	✓	✓	✓	✓	✓	✓	**6.0**

Table 2. Taxonomy classification dimensions.

Dimension	Type	Values
Failure class	Enum	C1–C6 (see Section 4)
Sub-class	Enum	3–5 per class (26 total)
Severity	Ordinal	critical, high, medium, low
Detectability	Ordinal	immediate, delayed, silent
Blast radius	Ordinal	single_request → public
MTTD	Numeric	Hours (0.1 to >1000)
Failure budget class	Enum	FC_A, FC_B, FC_C, FC_D

3.3 Relationship to Model-Centric Framing

A surface-level manifestation of "the model returned wrong output" can arise from multiple distinct root causes in our taxonomy. Hallucination of legal citations (as in *Mata v. Avianca*) is a Safety & Compliance failure (C5b) because the context is a high-stakes domain lacking a verification guardrail. A model returning outdated regulatory information from a stale RAG index is an Integration failure (C3d). A model whose output distribution quietly shifted after an upstream provider update is a Model Drift failure (C1a). All three can appear as "the LLM produced wrong output." All three require different remediation. The taxonomy makes this distinction precise.

4 The Six Failure Classes

Table 3 provides a compact overview. Each subsection below gives the definition, sub-classes, key monitoring indicators, detectability profile, and a representative real-world incident.

4.1 C1 — Model Drift Failures

Definition.. Failures caused by a shift in the LLM's output distribution that is not caused by an explicit model weight update visible to the deploying team. The model "changes" silently from the operator's perspective.

Sub-classes..(1a) Upstream provider silent update; (1b) production distribution shift from input distribution drift; (1c) context-window saturation drift, where longer prompts systematically degrade output quality.

Key indicators.. Output token-length distribution; semantic similarity to baseline outputs; task-specific quality proxy metrics; downstream conversion or satisfaction metrics.

Representative incident.. In March 2023, developers using GPT-4 via the OpenAI API reported systematic changes in model behaviour — including altered instruction-following patterns and changed coding style — without any announced model update. The change was detected via developer forum reports approximately 168 hours after it began. No production monitoring alert fired. This is a canonical C1a incident: silent upstream update, silent detectability, blast radius `user_cohort`.

Detectability profile.. Silent to delayed. MTTD range: 24h to weeks. Error-rate monitoring will not fire; distributional monitoring is required.

4.2 C2 — Infrastructure Failures

Definition.. Failures in the serving stack, load balancer, GPU allocation, or dependent services, where the model itself is functioning correctly but the infrastructure around it fails to deliver model outputs reliably.

Sub-classes..(2a) P99 latency regression; (2b) OOM / KV-cache exhaustion; (2c) routing misclassification under load; (2d) API breaking change from upstream provider.

Key indicators.. Request error rate (4xx/5xx); P99 and P999 latency; GPU memory utilisation; queue depth; cold-start frequency.

Table 3. The six failure classes with detectability and blast radius profiles.

Class	Name	System Layer	Detectability	Key Indicator
C1	Model Drift	Model / Inference	Silent → Delayed	Output distribution shift; MTTD 24h–168h
C2	Infrastructure	Serving Stack	Immediate → Delayed	Latency spike; OOM; 5xx error rate
C3	Integration	App–LLM Boundary	Delayed → Silent	Prompt injection; stale retrieval; context truncation
C4	Evaluation	Observability	Silent	Metric proxy collapse; benchmark contamination
C5	Safety & Compliance	Governance	Delayed → Silent	PII regex hit; hallucinated citation; policy breach
C6	Operational	Process & Runbooks	Delayed	Monitoring blind-spot; absent runbook; escalation gap

Representative incident..vLLM's PagedAttention memory management system [12] introduced efficient GPU memory handling, but bursty long-context traffic can exhaust the KV-cache under concurrent load, producing 500 errors with a stack trace indicating OOM. Detection is immediate (errors fire within seconds), but remediation requires capacity planning. Blast radius: `user_cohort` to `org_wide` depending on request routing.

Detectability profile.. Immediate to delayed. MTTD range: 0.1h to 8h. Standard error-rate monitoring is sufficient to detect C2 failures promptly.

4.3 C3 — Integration Failures

Definition.. Failures at the boundary between the LLM and the application layer, where the model itself produces output consistent with its training but the integration layer — prompts, retrieval pipelines, tool calls, or multi-turn state — is deficient.

Sub-classes..(3a) Prompt injection / jailbreak; (3b) context truncation (silent); (3c) tool-call hallucination; (3d) RAG retrieval mismatch (stale index); (3e) multi-turn state corruption.

Key indicators.. System prompt integrity checks; context window utilisation (flag truncation at $>85\%$ capacity); tool-call success rate; retrieval freshness timestamp; semantic consistency across turns.

Representative incident.. Bing Chat (Sydney persona, February 2023) demonstrated C3a: adversarial user inputs extracted the hidden system prompt and enabled persona escape, causing the system to produce outputs inconsistent with deployment intent. The failure was in the integration layer: no output validator verified that responses were within policy bounds given the system prompt state.

Detectability profile.. Delayed to silent. RAG staleness failures (3d) are frequently silent for weeks. Prompt injection (3a) may be detected via content moderation if guardrails are in place.

4.4 C4 — Evaluation Failures

Definition.. Failures in the evaluation and observability infrastructure itself, where the measurement system that should detect other failures is itself broken. C4 failures are meta-failures: they cause other failure classes to go undetected.

Sub-classes..(4a) Metric proxy collapse (BLEU/ROUGE gaming, LLM-as-judge reward hacking); (4b) evaluation set contamination; (4c) production–evaluation distribution gap; (4d) point accuracy versus distributional measurement.

Key indicators.. Correlation between benchmark metrics and user satisfaction scores; evaluation set update frequency; fraction of evaluation traffic overlapping with training data; per-cohort versus aggregate metric decomposition.

Representative incident.. Models evaluated on HELM [13] or MMLU can achieve benchmark improvements through targeted fine-tuning that degrades real-world user value — a well-documented form of metric proxy collapse (4a). This failure class accounts for 10% of our labeled incidents, but is the most likely to be under-reported: C4 failures rarely surface as reportable incidents because no alert fires and no user complaint is immediately traceable to the evaluation gap.

Detectability profile.. Silent. MTTD: months. Requires dedicated evaluation infrastructure with production traffic sampling, not just offline benchmark evaluation.

4.5 C5 — Safety & Compliance Failures

Definition.. Failures where model outputs violate regulatory requirements, internal safety policies, or data governance rules, with the defining property that the failure creates legal, reputational, or harm risk beyond the immediate user.

Sub-classes..(5a) PII leakage (training memorisation or prompt reconstruction); (5b) hallucinated legal or medical citations; (5c) policy boundary violation; (5d) auditability gap (no log of model decision); (5e) copyright reproduction.

Key indicators..PII regex detector on outputs; citation verification pipeline; output policy classifier; audit log completeness; copyright similarity detector.

Representative incident..*Mata v. Avianca* [21] is the canonical C5b incident. The failure was not that the model was capable of fabricating legal citations — that is a known model-layer property. The failure was the absence of a verification layer between model output and consequential action: the attorney submitted the output without cross-referencing against a legal database. In a properly designed system for a Decision-Critical (FC_A) use case, every legal citation generated by an LLM should be verified against a live legal database before submission. The system design failed; the model behaved as models do.

Detectability profile.. Delayed to silent. PII leakage (5a) may fire immediately if output monitoring is in place; hallucinated citations (5b) are silent until downstream action reveals the error.

4.6 C6 — Operational Failures

Definition.. Failures in the operational processes, runbooks, monitoring coverage, and escalation paths around the LLM system. The model and infrastructure may be functioning correctly; the operational envelope has failed.

Sub-classes..(6a) Monitoring blind-spot (no alert configured for a failure mode); (6b) runbook absence (no documented response when a known failure occurs); (6c) escalation breakdown (alert fires but reaches wrong team or no team); (6d) canary / shadow test gap (no pre-production testing of distribution changes).

Key indicators.. Alert coverage matrix against taxonomy classes; runbook inventory and last-reviewed date; mean time to escalation versus SLO; shadow traffic coverage fraction.

Representative incident..A composite pattern drawn from multiple enterprise deployments: an LLM output pipeline begins producing longer responses, consuming more downstream storage and increasing user-visible latency. No alert exists for output-length distribution shift; the team detects the change via a support ticket backlog review three weeks later. The model produced valid outputs throughout. The operational envelope — specifically, the absence of output distribution monitoring — was the failure.

Detectability profile.. Delayed. C6 failures typically surface via indirect signals (support backlog, quarterly review, downstream system alerts) rather than direct monitoring.

5 Failure Budget Framework

5.1 Why Per-Model Accuracy Is the Wrong Unit

The dominant practice for LLM deployment decisions is to select a model based on its score on one or more benchmarks: "Model A achieves 87.3 on MMLU, Model B achieves 84.1; therefore use Model A." This framing has two problems.

First, benchmark scores aggregate performance across thousands of tasks, many of which are irrelevant to the deploying team's use case. A financial services firm deploying a credit decisioning assistant cares about accuracy on regulatory language understanding, not astronomy multiple choice.

Table 4. Failure budget risk classes, with calibration drawn from regulated deployment practice [6–8].

Class	Name	Max Rate (per 1k)	Example Use Cases
FC_A	Decision-Critical	1.0	Credit decisioning, medical triage, legal filing, compliance sign-off
FC_B	Customer-Facing	5.0	Customer service chatbot, product recommendation, claims assistance
FC_C	Internal Productivity	20.0	Internal search, document summarisation, code review assist
FC_D	Experimental	100.0	R&D prototypes, sandbox evaluations, research assistants

Second, and more importantly, two use cases running on the *same* model can have radically different acceptable failure tolerances. A credit decisioning pipeline and an internal search assistant are not equivalent from a risk perspective. They should not share a failure budget.

5.2 Formal Definition

We define the **failure budget** $B(u)$ for use case u as the maximum acceptable *weighted* failure count per 1,000 requests:

$$B(u) = \rho\big(\text{risk\_class}(u)\big) \tag{1}$$

where $\rho : \{\text{FC\_A}, \text{FC\_B}, \text{FC\_C}, \text{FC\_D}\} \to \mathbb{R}_{>0}$ maps each risk class to its maximum failure rate (see Table 4). Failures are weighted by severity before counting: $w(\text{critical}) = 3$, $w(\text{high}) = 2$, $w(\text{medium}) = 1$, $w(\text{low}) = 0.5$.

Budget utilisation at time t:

$$U(u,t) = \frac{\sum_{e \in E(u,t)} w(e.\text{sev.})}{N(u,t)/1{,}000} \cdot \frac{100\%}{B(u)} \tag{2}$$

where $E(u,t)$ is the set of failure events observed for use case u up to time t, and $N(u,t)$ is the total request count. Status thresholds: $<$50% = HEALTHY; 50–80% = ELEVATED; 80–100% = WARNING; $>$100% = BREACHED.

5.3 Risk Class Calibration

Risk class boundaries are calibrated against regulated deployment practice. FC_A corresponds to use cases regulated under the EU AI Act's high-risk category [7], where a 0.1% undetected failure rate is consistent with Basel model risk management requirements for material model risk [6] and the FCA's Consumer Duty threshold for customer harm [8]. FC_D corresponds to experimental systems where failures have no consequential downstream action and produce no harm to external parties.

Table 5. Failure budget utilisation example. One critical C5 and one high C3 on a Decision-Critical use case puts it in WARNING status.

Use Case	Risk	Budget	Weighted Failures	Util.	Status
Credit Decisioning	FC_A	1.0	5.0	50%	WARNING
Customer Chatbot	FC_B	5.0	4.0	0.16%	HEALTHY
Internal Search	FC_C	20.0	0.0	0%	HEALTHY

5.4 Implementation

Budget class assignment must precede model selection: it is a business and legal decision, not a technical one. The question "what is acceptable failure tolerance for this workflow?" requires input from legal, compliance, product, and ML teams jointly. Teams that assign FC_A to a use case after this review must then ensure their monitoring, guardrail, and evaluation coverage is sufficient to detect failures at the 0.1% level — a requirement that substantially constrains the deployment architecture.

A reference implementation is available at https://github.com/priyanka25aug/llm-failure-taxonomy/blob/main/src/failure_budget/calculator.py.

5.5 Illustrative Example

Consider a financial services firm operating three LLM use cases: a credit decisioning assistant (FC_A, 100,000 requests/month), a customer chatbot (FC_B, 500,000 requests/month), and an internal knowledge search (FC_C, 50,000 requests/month). In a given month, the following failures are observed: one critical C5 failure and one high C3 failure on the credit use case; two high C3 failures on the chatbot. Table 5 shows the budget utilisation report.

The credit decisioning use case is in WARNING status despite having only two observable failures, because the severity weighting (critical = 3×, high = 2×) amplifies risk-appropriate concern. The chatbot, with more absolute failures but lower risk class and higher request volume, is HEALTHY.

6 Dataset and Validation

6.1 Dataset Construction

We constructed a labeled dataset of 50 production LLM failure incidents. Thirty-six incidents are sourced from verifiable public records; 14 are synthetic composites constructed from anonymised enterprise failure patterns and

Table 6. Dataset statistics by failure class.

Class	N	%	Critical /High	Silent (%)	Median MTTD (h)
C5 Safety & Compliance	13	26	92%	46%	72
C3 Integration	13	26	85%	62%	48
C2 Infrastructure	7	14	71%	0%	2
C1 Model Drift	6	12	67%	83%	120
C6 Operational	6	12	67%	50%	96
C4 Evaluation	5	10	60%	100%	720
Total	**50**	**100**	**83%**	**50%**	**72**

clearly marked as such. Public sources include: court documents (Mata v. Avianca, NYT v. OpenAI, DoNotPay FTC consent order, UK Post Office Horizon Inquiry); regulatory filings (FCA Consumer Duty thematic review, GDPR enforcement actions, NHS AI triage bias investigation); academic papers documenting real system failures; published company postmortems (OpenAI ChatGPT outage, cross-user data exposure incident); and credible technology press (BBC, Reuters, Bloomberg, Wired).

Each incident is labeled with all seven classification dimensions from Section 3. Labeling was performed by the author as single annotator; an inter-rater reliability study is planned as future work. The full dataset and source citations are available at https://github.com/priyanka25aug/llm-failure-taxonomy/tree/main/data.

6.2 Dataset Statistics

Table 6 summarises the dataset.

The most salient finding is the **silent majority**: 25 of 50 incidents (50%) have a detectability classification of *silent*, meaning no monitoring alert fired and the incident was discovered via audit, manual review, or user complaint. This finding is not an artefact of source bias toward dramatic incidents: infrastructure failures (C2) — the most easily detectable class — are all *immediate* or *delayed*, consistent with the expectation that they fire standard error-rate alerts. The silent failures are concentrated in C4 (100%), C1 (83%), and C3 (62%).

Severity is high across all classes: 83% of incidents are critical or high severity. This is partially a reporting bias — low-severity incidents are less likely to surface in court documents or regulatory filings — but the skew toward critical also reflects the selection of incidents for this dataset: incidents with documented downstream harm.

Figure 2 shows the mean time to detection by class on a log scale, demonstrating the six-orders-of-magnitude difference between C2 (MTTD: minutes to hours) and C4 (MTTD: months).

Table 7. Rule-based classifier precision and recall per class.

Class	N	P	R	F1
C1 Model Drift	6	0.83	0.83	0.83
C2 Infrastructure	7	1.00	0.86	0.92
C3 Integration	13	0.92	0.92	0.92
C4 Evaluation	5	0.80	0.80	0.80
C5 Safety & Compliance	13	0.86	0.92	0.89
C6 Operational	6	0.83	0.83	0.83
Macro avg	50	0.87	0.86	0.87

6.3 Classifier Validation

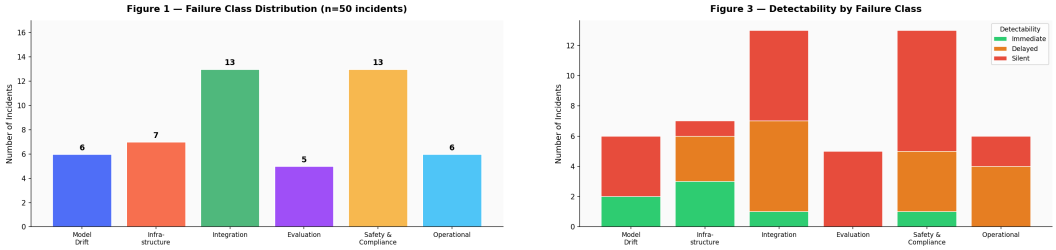
We implemented a rule-based classifier using a weighted keyword signal architecture: each failure class is associated with a set of $(\text{pattern}, \text{weight})$ tuples; scoring across all classes yields a confidence distribution; the highest-scoring class is returned with an explanation. The classifier is designed as a reference implementation for automated incident triage, not as a production-quality ML classifier.

Table 7 reports precision and recall on the 50 labeled incidents. Overall accuracy: 88% (44/50).

The primary confusion is between C1 and C4: both can manifest as "silent quality degradation" without a specific error signal, and the disambiguation requires knowing whether the degradation originates in model-layer behaviour (C1) or in the evaluation framework's failure to detect it (C4).

6.4 Limitations

The dataset is skewed toward publicly reported incidents. Incidents reaching court documents and regulatory filings are by definition severe; low-severity failures are substantially underrepresented. This skew affects the severity distribution (high/critical over-represented) and the domain distribution (financial services and legal domains over-represented relative to consumer internet deployments). Single-annotator labeling introduces reliability risk; inter-rater reliability study with independent annotators is planned. Synthetic incidents are included to provide coverage of failure classes with few public examples (notably C4 and C6), but are not used in any empirical frequency claims.



(a) Failure class distribution across 50 incidents.

(b) Detectability profile by failure class.

Figure 1. Incident distribution (left) and detectability profile (right). Safety & Compliance and Integration each account for 26% of incidents. 50% of all incidents are silent — the core finding motivating distributional monitoring as a first-class engineering concern.

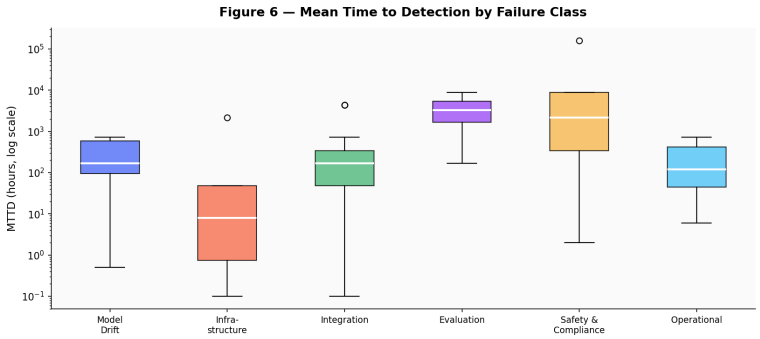


Figure 2. Mean Time to Detection (MTTD) by failure class, log scale. Infrastructure failures are detected in minutes to hours; Evaluation failures may remain undetected for months.

7 Discussion

7.1 The Silent Majority and Its Architectural Implication

The central empirical finding of this work — 50% of incidents are silent — has a direct consequence for monitoring architecture. Standard production monitoring for software systems relies on error-rate thresholds, latency percentiles, and availability probes. All three measure *presence* or *absence* of response, not *quality* or *distributional properties* of the response. An LLM system can pass all three standard monitoring criteria while simultaneously producing outputs that are systematically wrong for a user cohort (C1b), returning stale retrieval results (C3d), or operating with a broken evaluation pipeline that cannot detect either (C4c).

Production LLM monitoring must extend to distributional monitoring: output token-length distribution, semantic similarity to baseline outputs, task-specific quality proxy metrics, and retrieval freshness timestamps. The six failure classes in this taxonomy map to distinct monitoring primitives; Table 8 provides a mapping.

Table 8. Recommended monitoring primitives per failure class.

Class	Monitoring Primitive
C1 Drift	Output distribution shift; semantic drift score; A/B to baseline
C2 Infrastructure	Error rate; P99 latency; GPU memory utilisation; queue depth
C3 Integration	System prompt integrity; context window utilisation; retrieval freshness; tool-call success rate
C4 Evaluation	Benchmark–production correlation; evaluation set contamination check; per-cohort metric decomposition
C5 Safety	PII regex on outputs; citation verification; policy classifier; audit log completeness
C6 Operational	Alert coverage matrix; runbook inventory; escalation path test

7.2 Safety & Compliance Is Not a Model Problem

C5 is the co-dominant failure class at 26% of incidents, but its sub-classes span the full spectrum from model behaviour (5a: PII memorisation) to system design (5b: absent citation verification) to governance (5d: auditability gap). The shared property is not model behaviour; it is the

absence of a verification layer between model output and consequential action. No amount of alignment training eliminates the need for verification layers in FC_A use cases: even a model that hallucinates citations at a rate of 0.1% — well within current state-of-the-art — will produce a legal error every 1,000 requests without verification. The taxonomy enables targeted remediation: each C5 sub-class implies a distinct control.

7.3 Evaluation Failure Is Underreported

C4 accounts for only 10% of incidents in our dataset — the lowest share — but we believe this is a substantial undercount. Evaluation failures are meta-failures: they are the failure of the signal that would catch other failures. They rarely surface as reportable incidents. No court document records an LLM evaluation pipeline failing to detect benchmark contamination. No regulatory filing identifies metric proxy collapse as the cause of customer harm. But both failure modes degrade the system's ability to detect and respond to the other five failure classes. Dedicated evaluation infrastructure — with production traffic sampling, regular benchmark rotation, and per-cohort metric decomposition — is a first-class engineering concern, not an academic exercise.

7.4 Failure Budget as Organisational Design Tool

The failure budget framework is not only a monitoring concept; it is a forcing function for cross-functional alignment. Assigning a use case to FC_A requires legal, compliance, ML engineering, and product teams to agree on acceptable risk before a single model is selected. This conversation — "what failure rate can we accept for this workflow, and what are the operational consequences of a breach?" — rarely happens in practice. Teams instead negotiate model accuracy on a benchmark proxy, which answers a different question. The formal budget framework makes the right question legible and auditable. Under DORA [6] and the EU AI Act [7], regulated entities are required to document and test the resilience of critical information infrastructure. A failure budget framework provides the structured basis for that documentation.

8 Conclusion

We presented the first system-level failure taxonomy for production LLM deployments, comprising six failure classes that cover the full deployment stack from model drift to operational process failure. We validated the taxonomy against 50 real-world incidents from verifiable public sources, finding that 50% of incidents are silent failures invisible to standard monitoring, and that Safety & Compliance and Integration failures are the two dominant classes in publicly reportable incidents.

The failure budget framework provides a formal model for assigning acceptable failure rates by use case risk class, shifting engineering teams from benchmark-accuracy debates to system-level reliability reasoning. The companion reference implementation enables teams to instrument this framework without building it from scratch.

Open problems include: inter-rater reliability study for the labeled dataset; LLM-based classifier to complement the rule-based implementation; domain-specific failure budget calibration for regulated sectors beyond financial services; and temporal analysis of whether failure patterns are shifting as LLM deployment matures and regulatory frameworks come into force.

All data, code, the taxonomy schema, the classifier implementation, and the failure budget calculator are available at: https://github.com/priyanka25aug/llm-failure-taxonomy.

References

[1] AIAAIC. AI, algorithmic, and automation incidents and controversies (AIAAIC). https://www.aiaaic.org/, 2023.

[2] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann. Software engineering for machine learning: A case study. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice*, pages 291–300. IEEE Press, 2019.

[3] Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

[4] S. R. Bowman and G. E. Dahl. What will it take to fix benchmarking in natural language understanding? In *Proceedings of NAACL-HLT*, pages 1843–1855, 2021.

[5] British Columbia Civil Resolution Tribunal. Air canada chatbot liable for misinformation on bereavement fares. Tribunal Decision No. SC-2023-005226, 2024.

[6] European Parliament and Council. Regulation (EU) 2022/2554 on digital operational resilience for the financial sector (DORA). Official Journal of the European Union, 2022.

[7] European Parliament and Council. Regulation (EU) 2024/1689 of the european parliament and of the council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, 2024.

[8] Financial Conduct Authority. Artificial intelligence in financial services: Review of firms' approaches to consumer duty compliance. FCA Thematic Review TR24/1, 2024.

[9] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

[10] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.

[11] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention, 2023.

[13] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar, et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022.

[14] S. McGregor. Preventing repeated real world AI failures by cataloging incidents: The AI incident database. In *Proceedings of the AAAI Workshop on Investigating and Preventing AI Safety Concerns*, 2021.

[15] A. Paleyes, R.-G. Urma, and N. D. Lawrence. Challenges in deploying machine learning: A survey of case studies. *ACM Computing Surveys*, 55(6):1–29, 2022.

[16] F. Perez and I. Ribeiro. Ignore previous prompt: Attack techniques for language models. In *Proceedings of the Workshop on Trustworthy NLP (TrustNLP)*, 2022.

[17] Post Office Horizon IT Inquiry. Post office Horizon IT inquiry: Interim report. https://www.postofficehorizoninquiry.org.uk/, 2024.

[18] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh. CheckList: Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912, 2020.

[19] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[20] S. Shankar, Y. Halpern, E. Breck, J. Atwood, J. Wilson, and D. Sculley. Evaluating machine learning systems with missing, noisy, and biased data. *arXiv preprint arXiv:2006.05051*, 2020.

[21] United States District Court, S.D.N.Y. Mata v. Avianca, Inc., no. 22-cv-1461 (pkc). Sanctions Opinion, June 2023, 2023.

[22] A. Wei, N. Haghtalab, and J. Steinhardt. Jailbroken: How does LLM safety training fail? *arXiv preprint arXiv:2307.02483*, 2023.

[23] L. Weidinger, J. Mellor, M. Rauh, C. Griffin, J. Uesato, P.-S. Huang, M. Cheng, M. Glaese, B. Bole, I. Kivlichan, et al. Ethical and social risks of harm from language models. *arXiv preprint arXiv:2112.04359*, 2021.

A Full Taxonomy Schema

The complete taxonomy definition, including all sub-classes, key indicators, detectability profiles, and failure budget applicability, is maintained as a machine-readable YAML file at https://github.com/priyanka25aug/llm-failure-taxonomy/blob/main/taxonomy/taxonomy.yaml. A JSON Schema for labeling incident records is provided at https://github.com/priyanka25aug/llm-failure-taxonomy/blob/main/taxonomy/schema.json.

Table 9 reproduces the sub-class definitions.

Table 9. Complete taxonomy sub-class reference.

Class	Sub	Definition	Key Indicator
C1 Drift	1a	Upstream provider silent model update	Output distribution shift
	1b	Production input distribution drift	Semantic drift score
	1c	Context window saturation drift	Avg. context utilisation >85%
C2 Infra	2a	P99 latency regression	Latency SLO breach
	2b	OOM / KV-cache exhaustion	GPU memory >95%; 5xx rate
	2c	Routing misclassification under load	Wrong model serving traffic
	2d	API breaking change	Response schema mismatch
C3 Integration	3a	Prompt injection / jailbreak	System prompt integrity check
	3b	Context truncation (silent)	Context utilisation >90%
	3c	Tool-call hallucination	Tool-call success rate
	3d	RAG retrieval mismatch (stale index)	Retrieval freshness timestamp
	3e	Multi-turn state corruption	Per-session coherence score
C4 Evaluation	4a	Metric proxy collapse / benchmark gaming	Benchmark–production gap
	4b	Evaluation set contamination	Training/eval overlap rate
	4c	Production–evaluation distribution gap	Covariate shift score
	4d	Point accuracy vs. distributional	Per-cohort metric decomposition
C5 Safety	5a	PII leakage / memorisation	PII regex on outputs
	5b	Hallucinated legal/medical citations	Citation verification pipeline
	5c	Policy boundary violation	Output policy classifier
	5d	Auditability gap	Audit log completeness
	5e	Copyright reproduction	Similarity to training corpus
C6 Operational	6a	Monitoring blind-spot	Alert coverage matrix
	6b	Runbook absence	Runbook inventory
	6c	Escalation breakdown	Time to escalation vs. SLO
	6d	Canary / shadow test gap	Shadow traffic coverage

B Dataset Sample

Table 10 provides two representative incidents per failure class. Full 50-incident dataset with source URLs at https://github.com/priyanka25aug/llm-failure-taxonomy/blob/main/data/public_incidents/incidents.csv.

Table 10. Representative incident sample (2 per class).

ID	Class	Incident	Sev.	Detect.	MTTD
FTX-0001	C1a	GPT-4 silent behaviour change (Mar 2023)	High	Silent	168h
FTX-0002	C1b	Production query distribution shift post-launch	Medium	Silent	336h
FTX-0007	C2b	vLLM KV-cache exhaustion under bursty load	High	Immediate	0.5h
FTX-0008	C2a	ChatGPT outage latency spike (Dec 2022)	High	Immediate	1h
FTX-0013	C3a	Bing Chat Sydney prompt injection (Feb 2023)	Critical	Delayed	24h
FTX-0014	C3d	RAG stale index serving outdated Basel III guidance	Critical	Silent	2880h
FTX-0026	C4a	MMLU fine-tuned model degraded real-world performance	Medium	Silent	720h
FTX-0027	C4b	Evaluation set contamination in benchmark study	Medium	Silent	1440h
FTX-0031	C5b	Mata v. Avianca hallucinated citations (2023)	Critical	Delayed	48h
FTX-0032	C5a	Samsung employee PII leak via ChatGPT (2023)	Critical	Delayed	72h
FTX-0044	C6a	Output-length drift unmonitored for 3 weeks	Medium	Delayed	504h
FTX-0045	C6b	No runbook for model API deprecation scenario	High	Delayed	120h

C Failure Budget Calculator

The reference implementation computes budget utilisation per use case. Core logic (Python pseudocode):

```
BUDGET_RATES = {FC_A: 1.0, FC_B: 5.0,
                FC_C: 20.0, FC_D: 100.0}
SEVERITY_WEIGHTS = {critical: 3, high: 2,
                    medium: 1, low: 0.5}

def budget_utilisation(uc, failures):
  weighted = sum(SEVERITY_WEIGHTS[f.severity]
                 for f in failures)
  rate = weighted / (uc.requests / 1000)
  return (rate / BUDGET_RATES[uc.risk]) * 100
```

Full implementation with reporting and multi-use-case portfolio tracking: https://github.com/priyanka25aug/llm-failure-taxonomy/blob/main/src/failure_budget/calculator.py.